

Flash Data Storage

I. Experiment Objective

Learn how to use the NVS (Non-Volatile Storage) component on the ESP32-S3 platform to implement persistent data storage. Master the method of storing and reading different data types (int8, uint16, int32, blob) through key-value pairs, as well as the experiment flow of triggering data writes with a button.

II. Experiment Principle

1. Keywords

Keyword	Explanation
NVS (Non-Volatile Storage)	A key-value storage component provided by ESP-IDF. Data is saved in the NVS partition of Flash and is not lost after power-off. It is operated through the API flow of <code>nvs_open</code> → <code>nvs_set_xxx</code> / <code>nvs_get_xxx</code> → <code>nvs_commit</code> → <code>nvs_close</code> .
Flash memory	The SPI Flash chip onboard the ESP32-S3, used to store the firmware program and data. NVS carves out a dedicated partition in Flash (about 24KB by default) for persistently saving small amounts of data such as configuration parameters.
Key-Value pair	NVS uses a string as the key (e.g., "TEST_UINT16") and supports multiple basic types or binary blocks as the value. It supports data types such as int8, uint8, int16, uint16, int32, uint32, int64, uint64, string, and blob.
Blob type	Binary Large Object, used to store arbitrary binary data such as strings and structs. In this experiment, a blob is used to save a string such as "blob:2". The blob size is limited by the remaining space of the NVS partition.
Namespace	NVS uses namespaces (e.g., "YAHBOOM") to isolate key-value pairs of different modules and avoid key name conflicts. Fields with the same name under different namespaces can be stored independently.

2. Technical Architecture

The program consists of two components and one main program. It triggers incremental data writes through a button and verifies the power-off retention capability.

```
main/main.c
├─ Flash_Init() / Flash_Read_Data_xxx() / Flash_Write_Data_xxx()
│   └─ components/flash/      NVS storage encapsulation layer
│       └─ flash.h / flash.c  Read/write encapsulation based on the ESP-IDF NVS API
│       └─ STORAGE_NAMESPACE = "YAHBOOM"
├─ Key_Init() / Key1_Read_State()
│   └─ components/key/        Button driver
│       └─ key.h / key.c      GPIO42 polling + software debounce
```

The main loop detects a button press → increments the values → calls the Flash write interface (with read-back verification) → prints the current values over the serial port. After a power-off and power-on, the persistent data in Flash is automatically restored.

III. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB data cable (Type-C)	✓ Required

2. Prerequisites

1. Set up the ESP32-IDF development environment
2. Connect the MicroROS-V2 control board to the computer via USB

3. Compile and Flash

```
cd ~/esp/base_samples/flash
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

To exit the monitor: press `Ctrl + ]`.

4. Stopping Method

In the terminal, press `Ctrl+]` to exit the serial monitor.

IV. Experiment Result

After flashing and running the program, you can see the following output in the serial monitor terminal:

First power-on (NVS partition data is empty):

```
hello yahboom
I (1102) FLASH_MAIN: Nice to meet you!
I (1112) FLASH_MAIN: read u16 value: 0
I (1112) FLASH_MAIN: read i32 value: 0
I (1112) FLASH_MAIN: read blob:
```

After pressing button KEY1:

```
I (1212) FLASH_MAIN: KEY 1 PRESS
I (1212) FLASH_MAIN: save u16 value: 1
I (1212) FLASH_MAIN: save i32 value: 1
I (1212) FLASH_MAIN: save flash blob: blob:1
```

Pressing button KEY1 again:

```
I (1312) FLASH_MAIN: KEY 1 PRESS
I (1312) FLASH_MAIN: save u16 value: 2
I (1312) FLASH_MAIN: save i32 value: 2
I (1312) FLASH_MAIN: save flash blob: blob:2
```

After power-off and power-on again:

```
hello yahboom
I (1102) FLASH_MAIN: Nice to meet you!
I (1112) FLASH_MAIN: read u16 value: 2
I (1112) FLASH_MAIN: read i32 value: 2
I (1112) FLASH_MAIN: read blob: blob:2
```

It can be seen that the data before power-off has been successfully saved and restored.

V. Program Analysis

Source code paths:

- Firmware source: `~/esp/base_samples/flash/main/main.c`
- Flash storage: `~/esp/base_samples/flash/components/flash/flash.h`, `flash.c`
- Button driver: `~/esp/base_samples/flash/components/key/key.h`, `key.c`

1. Directory Structure

```
flash/
├─ CMakeLists.txt
├─ main/
│   ├─ CMakeLists.txt
│   └─ main.c
└─ components/
    ├─ flash/          # NVS storage encapsulation
    │   ├─ flash.h
    │   └─ flash.c
    └─ key/            # Button driver
        ├─ key.h
        └─ key.c
```

2. Main Program Logic

`main/main.c` implements the Flash data read/write function:

1. Initialize the button and Flash/NVS subsystems
2. Read the three types of data saved last time from Flash and print them:
 - uint16 value (key: `TEST_UINT16`)
 - int32 value (key: `TEST_INT32`)
 - blob string (key: `TEST_BLOB`)
3. The main loop detects the button state:
 - Each time the button (GPIO42) is pressed, the three values are incremented and re-saved to Flash
 - The blob string is synchronously updated to `"blob:current value"` and saved

```
Key_Init();
Flash_Init();

// Read the saved data
Flash_Read_Data_UInt16(FKEY_TEST_UINT16, &value_u16);
Flash_Read_Data_Int32(FKEY_TEST_INT32, &value_i32);
Flash_Read_Blob(FKEY_TEST_BLOB, test_blob, &blob_len);

while (1) {
    if (Key1_Read_State() == KEY_STATE_PRESS) {
        value_u16++;
        value_i32++;
        Flash_Write_Data_UInt16(FKEY_TEST_UINT16, value_u16);
        Flash_Write_Data_Int32(FKEY_TEST_INT32, value_i32);
        sprintf(test_blob, "blob:%ld", value_i32);
        Flash_Write_Blob(FKEY_TEST_BLOB, test_blob, blob_len);
    }
    vTaskDelay(pdMS_TO_TICKS(10));
}
```

3. Key Components

Flash storage encapsulation

Files: `components/flash/flash.h`, `components/flash/flash.c`

Implemented based on the ESP-IDF NVS (Non-Volatile Storage) library, saving data in the NVS partition of Flash. The data is not lost after power-off.

Storage parameters:

Parameter definition file: `~/esp/base_samples/flash/components/flash/flash.h`

Macro	Value	Description
<code>STORAGE_NAMESPACE</code>	"YAHBOOM"	NVS namespace; all key-value pairs are managed under this namespace

Key name definitions:

Macro	Key Name String	Data Type	Initial Value
<code>FKEY_TEST_UINT16</code>	"TEST_UINT16"	uint16	0
<code>FKEY_TEST_INT32</code>	"TEST_INT32"	int32	0
<code>FKEY_TEST_BLOB</code>	"TEST_BLOB"	blob (string)	""

NVS read/write flow (taking uint16 write as an example):

```

esp_err_t Flash_Write_Data_UInt16(const char* key, uint16_t value)
{
    nvs_handle_t my_handle;
    esp_err_t err;
    uint16_t read_value = 0;

    // 1. Open the NVS namespace (read/write mode)
    err = nvs_open(STORAGE_NAMESPACE, NVS_READWRITE, &my_handle);
    if (err != ESP_OK) return err;

    // 2. Write the key-value pair
    err = nvs_set_u16(my_handle, key, value);

    // 3. Commit the write (must be called, otherwise the data is not actually written to
    Flash)
    if (err == ESP_OK) err = nvs_commit(my_handle);

    // 4. Read back for verification
    if (err == ESP_OK) err = nvs_get_u16(my_handle, key, &read_value);
    if (err == ESP_OK && value != read_value) err = ESP_FAIL;

    // 5. Close the NVS handle
    nvs_close(my_handle);
    return err;
}

```

Initialization function `Flash_Init()`:

```

void Flash_Init(void)
{
    esp_err_t err = nvs_flash_init();
    if (err == ESP_ERR_NVS_NO_FREE_PAGES || err == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        // The NVS partition has insufficient space or a version change; erase and retry
        ESP_ERROR_CHECK(nvs_flash_erase());
        err = nvs_flash_init();
    }
    ESP_ERROR_CHECK(err);
}

```

Supported read/write interfaces:

Function	Description
<code>Flash_Read_Data_Int8(key, &value)</code>	Reads int8 data
<code>Flash_write_Data_Int8(key, value)</code>	Writes int8 data
<code>Flash_Read_Data_UInt16(key, &value)</code>	Reads uint16 data
<code>Flash_write_Data_UInt16(key, value)</code>	Writes uint16 data
<code>Flash_Read_Data_Int32(key, &value)</code>	Reads int32 data
<code>Flash_write_Data_Int32(key, value)</code>	Writes int32 data
<code>Flash_Read_Blob(key, data, &length)</code>	Reads blob binary data
<code>Flash_write_Blob(key, data, length)</code>	Writes blob binary data

NVS API description:

All read/write interfaces follow the same pattern: `nvs_open` -> `nvs_set_XXX` / `nvs_get_XXX` -> `nvs_commit` (required for writes) -> `nvs_close`.

- `nvs_set_u16(handle, key, value)` — Sets a uint16 value
- `nvs_get_i32(handle, key, &value)` — Gets an int32 value
- `nvs_set_blob(handle, key, data, length)` — Sets binary data
- `nvs_get_blob(handle, key, buffer, &length)` — Gets binary data; when `length` is input, it is the buffer size, and when output, it returns the actual data size
- `nvs_commit(handle)` — Commits the changes to ensure the data is written to Flash

Note: NVS write operations have a lifetime limit (usually about 100,000 erase/write cycles), so frequent writes should avoid excessively large data volumes. Blob data is limited by the NVS partition size (the default NVS partition is usually 24KB).

Button driver

Files: `components/key/key.h`, `components/key/key.c`

Button pin:

Button	GPIO	Description
KEY1 (USER1)	GPIO42	User button; low level when pressed

Configuration parameters:

- Mode: input mode (`GPIO_MODE_INPUT`)
- Pull-up: internal pull-up enabled (`GPIO_PULLUP_ENABLE`)
- Trigger: no interrupt; the level is read through polling

Core functions:

- `key_Init()` — Initializes the button GPIO (configures GPIO42 as input with pull-up)
- `key1_Read_State()` — Non-blocking button state reading with software debounce; returns `KEY_STATE_PRESS` once per press

Debounce mechanism:

Software debounce is implemented with a state machine counter:

- After detecting low level 3 consecutive times, a press event is returned on the 2nd detection (`key_state == 2`)
- When a high level is detected, the counter resets to zero, ensuring that pressing again after release can re-trigger
- This mechanism eliminates the jitter interference of the mechanical button while avoiding repeated triggering on long press

VI. Common Problems

Problem	Cause	Solution
Data reads as 0	NVS partition not initialized or data not written	Check whether <code>Flash_Init()</code> is called successfully and confirm that <code>nvs_flash_init()</code> returns ESP_OK
Button does not respond	Abnormal GPIO42 connection or pull-up not enabled	Check the button pin and confirm that the pull-up configuration is enabled
Data lost after write and reboot	<code>nvs_commit()</code> not called	Confirm that the <code>Flash_write_*</code> functions call <code>nvs_commit()</code> to commit the changes
Blob data truncated	The input buffer length is insufficient	Enlarge the read buffer; the <code>length</code> parameter of <code>Flash_Read_Blob()</code> returns the actual data length

Problem	Cause	Solution
NVS initialization failed	The Flash partition table does not contain an NVS partition	Confirm that <code>partition.csv</code> contains the NVS partition configuration